
Component-wise Gradient Boosting for Distributional Regression in R

Term Paper (asp26boost)

Bjarne Herbst

`bjarne.herbst01@stud.uni-goettingen.de`

Luca Messerschmidt

`j.messerschmidt03@stud.uni-goettingen.de`

Benedict Schnitzler

`b.schnitzler@stud.uni-goettingen.de`

September 14, 2026

Term paper for Module M.WIWI-QMW.0011

(Advanced Statistical Programming with R)

Person responsible for the module: Prof. Dr. Elisabeth Bergherr

Georg-August-Universität Göttingen

Contents

1	Introduction	1
2	Theoretical Background	2
2.1	Distributional regression and additive predictors	2
2.2	Component-wise gradient boosting	3
2.3	Base learners and regularization	4
3	Implementation and Usage	7
3.1	Package architecture and data preparation	9
3.2	Candidate learners and the boosting loop	10
3.3	Tuning and package interface	12
4	Simulation Study	14
4.1	Study design	14
4.2	Results	21
5	Conclusion	24
	References	26
A	Additional figures and tables	29
B	Convergence plots	30
C	Reproducibility	32
D	Package manual	33
E	Declaration of originality	59

1 Introduction

Many regression models describe how the conditional mean changes with the covariates while other parameters of the response distribution remain fixed. This can be inadequate when the variability or shape of the distribution also depends on the covariates. Generalized additive models for location, scale and shape (GAMLSS) allow each modeled distributional parameter to be linked to the covariates through its own additive predictor [Rigby and Stasinopoulos, 2005]. The R package `gamlss` provides an implementation of this framework [Stasinopoulos and Rigby, 2007].

Classical GAMLSS are fitted by penalized maximum likelihood and variable selection usually relies on information criteria. These procedures can be unstable and become infeasible in high-dimensional settings, especially when the number of candidate predictors exceeds the number of observations [Mayr et al., 2012a]. Component-wise gradient boosting offers a fitting method for such settings. It treats estimation as functional gradient descent and updates the model in small steps [Friedman, 2001, Bühlmann and Hothorn, 2007]. For each parameter update, candidate base learners are fitted to the current negative gradient of the loss. Only the best-fitting learner is used to update the corresponding additive predictor [Bühlmann and Yu, 2003, Bühlmann and Hothorn, 2007]. Hothorn et al. [2010] describe an implementation of this framework for structured additive regression in the R package `mboost`. Mayr et al. [2012a] adapt the method to GAMLSS. Their algorithm updates the parameter-specific predictors successively with respect to a common likelihood. The corresponding software is provided by the R package `gamboostLSS` [Hofner et al., 2016]. Since the updates are component-wise and the procedure is stopped early, effect estimates are shrunk and some variables may never enter the model [Mayr et al., 2012a, Hofner et al., 2016].

We developed `asp26boost` [Messerschmidt et al., 2026], an R package [R Core Team, 2024] that implements component-wise gradient boosting for GAMLSS-type distributional regression. Its main application is Gaussian location-scale regression. The same fitting engine also supports Gamma, Poisson and Binomial families, with the Binomial family restricted to binary responses. The package provides linear and P-spline base learners as well as cyclic and noncyclic parameter updates. The aim of this paper is to explain the statistical basis of the method and show how it is implemented in `asp26boost`. A simulation study then examines the package’s variable selection,

computational performance and ability to recover nonlinear effects.

Section 2 introduces distributional regression, likelihood-based boosting, base learners, parameter update schedules and early stopping. Section 3 describes the package architecture, fitting loop, interface and tuning procedures. Section 4 presents the simulation study. Section 5 summarizes the findings and discusses limitations and possible extensions.

2 Theoretical Background

This section develops the statistical basis of `asp26boost`. It first introduces distributional regression and additive predictors. It then describes component-wise likelihood-based boosting, linear and P-spline base learners and the update schedules for models with several distributional parameters. The final part explains early stopping as a form of regularization.

2.1 Distributional regression and additive predictors

Distributional regression allows several parameters of the response distribution to depend on covariates. For observations $i = 1, \dots, n$, let y_i denote the response and $\mathbf{x}_i$ the corresponding covariate vector. Given the covariates, the observations are assumed to be independent and y_i has conditional density $f(y_i | \boldsymbol{\theta}_i)$. The vector

$$\boldsymbol{\theta}_i = (\theta_{i1}, \dots, \theta_{iK})^\top$$

contains the K distributional parameters at observation i . Each component θ_{ik} may depend on the covariates through a separate additive predictor η_k , whose value at observation i is denoted by η_{ik} [Rigby and Stasinopoulos, 2005, Mayr et al., 2012a]. The number and interpretation of the parameters depend on the chosen response distribution.

A parameter-specific link function g_k relates θ_{ik} to its additive predictor:

$$g_k(\theta_{ik}) = \eta_{ik} = \beta_{0k} + \sum_{j=1}^{p_k} f_{jk}(x_{ij}), \quad k = 1, \dots, K. \quad (1)$$

Here, β_{0k} is the intercept and p_k is the number of terms in the predictor for parameter k .

The function f_{jk} describes the effect of covariate j on that parameter. In `asp26boost`, this effect is represented by either a linear or a P-spline base learner. Both the covariates and the forms of their effects may differ between parameters. Fitted parameter values are obtained through the inverse link,

$$\theta_{ik} = g_k^{-1}(\eta_{ik}).$$

For example, a log link maps a positive parameter to the real line so its inverse produces positive fitted values.

The following sections retain the generic notation $f(y_i \mid \boldsymbol{\theta}_i)$.

2.2 Component-wise gradient boosting

Assuming conditional independence across observations, model fitting can be expressed as minimization of the empirical risk

$$R = \sum_{i=1}^n \rho(y_i, \boldsymbol{\eta}_i) = - \sum_{i=1}^n \log f(y_i \mid \boldsymbol{\theta}_i), \quad (2)$$

where $\boldsymbol{\eta}_i = (\eta_{i1}, \dots, \eta_{iK})^\top$ and ρ is the negative log-likelihood contribution of observation i . Minimizing R is equivalent to maximizing the likelihood. Component-wise gradient boosting approaches this problem through a sequence of small updates instead of estimating all effects at once [Friedman, 2001, Mayr et al., 2012a].

For a prospective update of parameter k , let $\hat{\boldsymbol{\eta}}_i$ denote the current fitted predictor vector. The working response is the negative partial gradient

$$u_{ik} = - \left. \frac{\partial \rho(y_i, \boldsymbol{\eta}_i)}{\partial \eta_{ik}} \right|_{\boldsymbol{\eta}_i = \hat{\boldsymbol{\eta}}_i} = \left. \frac{\partial \log f(y_i \mid \boldsymbol{\theta}_i)}{\partial \eta_{ik}} \right|_{\boldsymbol{\eta}_i = \hat{\boldsymbol{\eta}}_i}. \quad (3)$$

The derivative includes the inverse link $\theta_{ik} = g_k^{-1}(\eta_{ik})$. The values u_{ik} are therefore scores on the predictor scale. They serve as temporary responses when the base learners are fitted and coincide with ordinary residuals only for particular loss functions [Friedman, 2001, Bühlmann and Hothorn, 2007]. Since the likelihood depends on the complete parameter vector $\boldsymbol{\theta}_i$, the gradient for one parameter may depend on the current estimates of the others.

For parameter k , each of the L_k candidate base learners is fitted to the working response

vector $\mathbf{u}_k = (u_{1k}, \dots, u_{nk})^\top$. After fitting the candidates, the learner with the smallest residual sum of squares is selected:

$$\ell_k^* = \arg \min_{\ell=1, \dots, L_k} \sum_{i=1}^n \left[u_{ik} - \hat{h}_{\ell k}(\mathbf{x}_i) \right]^2. \quad (4)$$

Only the selected learner $\hat{h}_k^* = \hat{h}_{\ell_k^* k}$ contributes to the update [Bühlmann and Yu, 2003, Hothorn et al., 2010, Mayr et al., 2012a]:

$$\hat{\eta}_{ik}^{\text{new}} = \hat{\eta}_{ik}^{\text{old}} + \nu_k \hat{h}_k^*(\mathbf{x}_i), \quad 0 < \nu_k \leq 1. \quad (5)$$

The step size ν_k controls how much of the fitted learner is added. The updated distributional parameter is obtained through g_k^{-1} . Section 2.3 explains how the package fits the individual base learners and how it decides which parameter update is applied. Repeating these steps produces a sequence of increasingly complex models. Covariate effects that have not been selected remain zero. Stopping the procedure before every candidate has entered can therefore produce a sparse model while shrinking the effects that have already been selected [Hothorn et al., 2010, Mayr et al., 2012a,b]. The role of the stopping iteration is discussed in Section 2.3.

2.3 Base learners and regularization

Section 2.2 described a single boosting update. Three choices remain: which base learners may be selected, how updates are divided among the distributional parameters and where the boosting path is stopped.

P-spline base learners For a covariate x_j , a linear base learner can only represent a straight-line effect. To allow for nonlinear effects, `asp26boost` also provides P-spline base learners. The effect of x_j on parameter k is represented as

$$f_{jk}(x_{ij}) = \mathbf{B}_j(x_{ij})^\top \boldsymbol{\gamma}_{jk},$$

where $\mathbf{B}_j(x_{ij})$ is a cubic B-spline basis and $\boldsymbol{\gamma}_{jk}$ is the corresponding coefficient vector. The P-spline base learner combines this B-spline basis with a difference penalty on the coefficients [Eilers and Marx, 1996]. Its internal knots are placed at empirical quantiles

of x_j .¹

For the working gradient $\mathbf{u}_k$ from Section 2.2, the B-spline coefficients are estimated by

$$\hat{\gamma}_{jk} = \arg \min_{\gamma} \{ \|\mathbf{u}_k - \mathbf{B}_j \gamma\|_2^2 + \lambda_j \gamma^\top \mathbf{K}_j \gamma \}.$$

Here, $\mathbf{K}_j = \mathbf{D}_2^\top \mathbf{D}_2$ is based on second differences of neighboring coefficients. The penalty therefore controls the roughness of the fitted effect rather than the size of the coefficients themselves. A larger λ_j produces a smoother fit while a smaller value allows more variation [Eilers and Marx, 1996].

Base learners with greater flexibility tend to achieve a lower residual sum of squares even when they contain no stronger signal. This can bias componentwise selection toward more flexible candidates [Schmid and Hothorn, 2008]. The flexibility of a P-spline can be measured by its effective degrees of freedom,

$$\text{df}_j(\lambda_j) = \text{tr} \left[\mathbf{B}_j (\mathbf{B}_j^\top \mathbf{B}_j + \lambda_j \mathbf{K}_j)^{-1} \mathbf{B}_j^\top \right].$$

`asp26boost` selects λ_j so that every P-spline candidate has a target of four effective degrees of freedom.² Since this quantity depends only on the basis and penalty matrices, the required value of λ_j can be found before boosting begins.

This adjustment makes the P-spline candidates comparable with each other. It does not give linear and P-spline candidates the same flexibility because a linear candidate with an intercept and slope has two degrees of freedom. In `learner = "auto"` mode, a P-spline may therefore be selected partly because it is more flexible. The settings `"linear"` and `"spline"` avoid this comparison by restricting all candidates to one learner type.

Parameter update schedules With K distributional parameters, the algorithm must also decide which parameter to update in each round. `asp26boost` provides cyclic and noncyclic updating.

Under the cyclic algorithm of Mayr et al. [2012a], every active parameter is updated once per round in a fixed order. Before updating parameter k , the algorithm evaluates its current negative gradient, selects the best-fitting base learner and adds the resulting

¹See `R/learner_spline.R`, line 27.

²See `R/learner_spline.R`, lines 72–82.

step to η_k . Since the parameters are updated in sequence, later updates in the same round use the parameter estimates produced by earlier updates. Each parameter may have its own stopping iteration $m_{\text{stop},k}$ and is no longer updated after reaching it.

The noncyclic algorithm instead forms the best candidate update for every parameter and calculates the total empirical risk that would result from each candidate. It applies only the candidate with the smallest resulting risk [Thomas et al., 2018]. Exactly one parameter is therefore updated in each round. In `asp26boost`, the noncyclic algorithm uses one shared stopping iteration for the complete model.

Both schedules use the same gradients and base learners. They differ in how updates are allocated across the distributional parameters.

Step size and early stopping The boosting path is controlled by the step size ν_k and the stopping iteration. The step size determines how much of the selected base learner is added in one update. Smaller values produce finer changes but require more iterations. A value of $\nu_k = 0.1$ is common in statistical boosting and is also the default in comparable implementations [Schmid and Hothorn, 2008, Thomas et al., 2018]. Different step sizes can be specified for the distributional parameters in `asp26boost`. Its grid-search function can also compare several values.

The stopping iteration is the main regularization parameter of the model [Mayr et al., 2012b]. At the start of the path, the predictors contain only their offset values. Each subsequent update weakens the regularization by adding another fitted component. Under suitable conditions, a sufficiently long path can approach the corresponding unregularized maximum likelihood fit [Bühlmann and Hothorn, 2007, Mayr et al., 2012a]. Boosting is usually stopped before reaching that point. Early stopping shrinks effects toward their initial values, which can reduce variance at the cost of some bias. It also produces variable selection because base learners that have not been selected by m_{stop} do not contribute to the fitted predictor [Mayr et al., 2012a,b].

Training risk is not an independent measure of predictive performance because the same observations are used to construct every update. The stopping iteration should therefore be selected from predictive risk evaluated on held-out observations, usually through cross-validation or another resampling method [Mayr et al., 2012a].

For cyclic boosting, each parameter may in principle have a separate stopping iteration.

Selecting the vector

$$\mathbf{m}_{\text{stop}} = (m_{\text{stop},1}, \dots, m_{\text{stop},K})$$

then requires a multidimensional search [Mayr et al., 2012a]. `asp26boost` accepts separate stopping budgets when a cyclic model is fitted directly. Its current cross-validation functions, however, search only over one common stopping iteration for the two Gaussian predictors.

3 Implementation and Usage

The package provides separate fitting functions for Gaussian, Gamma, Poisson and binomial responses. For example, a Gaussian location-scale model can be fitted as follows:

```
fit <- boost_gaussian(  
  X = X,  
  Z = Z,  
  y = y,  
  mstop = 100,  
  nu_mu = 0.1,  
  nu_sigma = 0.1  
)  
  
print(fit)  
  
predict(  
  fit,  
  newdata_X = X[1:5, ],  
  newdata_Z = Z[1:5, ]  
)
```

Here, `X` contains the covariates for the mean predictor and `Z` those for the scale predictor. The fitting functions differ in their response-specific checks and result processing, but all four eventually call the same generalized boosting loop. Figure 1 summarizes this common workflow.

The package includes an automated test suite implemented with `testthat`. At the

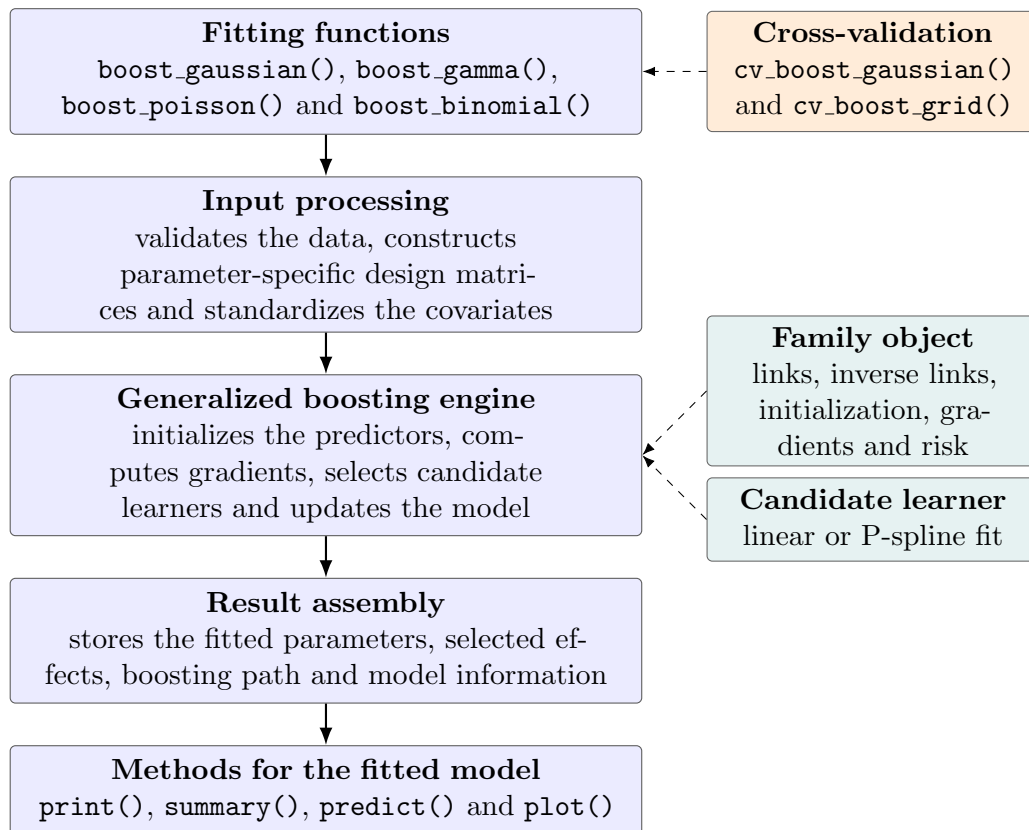


Figure 1: High-level fitting workflow in `asp26boost`. Solid arrows show the path from a fitting call to the fitted model and its methods. Dashed arrows indicate repeated fitting during cross-validation and components used by the boosting engine.

time of writing all tests passed, and `covr` reported a code coverage of 84.9% [Hester, 2023]. Its runtime dependencies are limited to `stats`, `graphics` and `splines`, all of which are distributed with R [R Core Team, 2024].

3.1 Package architecture and data preparation

`asp26boost` separates the parts that depend on the response distribution from the general fitting algorithm through a family object: a named list containing the distributional parameters and their update order, the link and inverse-link functions, starting values, one negative-gradient function per parameter and the empirical risk. The boosting loop accesses these components through a common interface and does not contain separate branches for the four response distributions, following the same design principle as the family infrastructure in `mboost` and its extension to several distributional parameters in `gamboostLSS` [Hothorn et al., 2010, Hofner et al., 2016].

The four public fitting functions create the appropriate family object and pass it to the same generalized boosting loop together with the parameter-specific design matrices, step sizes and stopping budgets. The Gaussian implementation retains its own input processing and result structure to preserve the package’s original interface. The Gamma, Poisson and Binomial functions share a common wrapper for validation, data preparation and result assembly. This difference concerns the code surrounding the boosting loop, not the fitting algorithm itself.

Models can be specified either with numeric matrices or data frames, one design per distributional parameter, or through a formula interface. A single-parameter model takes one two-sided formula containing the response; a two-parameter model takes a named list of formulas, the first two-sided to supply the response and the second one-sided. Both interfaces are converted into numeric design matrices with `model.frame()` and `model.matrix()`, with the automatically generated intercept column removed, so they produce the same internal representation before fitting begins.

The package then checks that each design has the same number of observations as the response and contains only finite numeric values without missing observations. It also validates the step sizes, stopping budgets and update schedule. Further checks that depend on the response distribution are handled by the corresponding public fitting function.

Each parameter-specific design is standardized separately. For a predictor column x_j , the transformed values are

$$x_{ij}^* = \frac{x_{ij} - \bar{x}_j}{s_j},$$

where $\bar{x}_j$ and s_j are the sample mean and sample standard deviation. A zero standard deviation is replaced by one, which maps a constant column to zero. Centering is required by the simplified least-squares calculation used for linear base learners in Section 3.2. Scaling does not change the fitted values or residual sum of squares of a linear candidate, but it keeps the numerical scale of the design matrices consistent. The means and standard deviations are stored with the fitted model and applied again when predictions are made for new data. Accumulated linear coefficients are also converted back to the original predictor scale before they are reported. P-spline effects are retained through their B-spline basis representation rather than summarized by a single transformed coefficient.

After data preparation, the family object supplies a constant starting value for every parameter. For the Gaussian family, the initial predictors are

$$\eta_{i,\mu}^{[0]} = \bar{y} \quad \text{and} \quad \eta_{i,\sigma}^{[0]} = \log(s_y),$$

where s_y is the sample standard deviation of the response. These values form the initial model from which boosting starts. Although the intercept columns created by `model.matrix()` are removed, the overall level of each predictor is not fixed. Every linear candidate estimates a constant adjustment from the mean working gradient, and the B-spline basis can also represent a constant component. These adjustments accumulate over the boosting path.

3.2 Candidate learners and the boosting loop

At each parameter update, the fitting engine obtains the current working gradient $\mathbf{u}_k = (u_{1k}, \dots, u_{nk})^\top$ from `family$ngradient[[k]]`. The family object's risk function returns the corresponding total negative log-likelihood. These functions have the same interface for every response family, so the remaining fitting procedure is shared.

For a linear candidate based on the standardized covariate x_{ij}^* , the package computes

$$\widehat{a}_k = \bar{u}_k \quad \text{and} \quad \widehat{b}_{jk} = \frac{\sum_{i=1}^n x_{ij}^* u_{ik}}{\sum_{i=1}^n (x_{ij}^*)^2}.$$

Since x_j^* is centered, these are the ordinary least-squares intercept and slope. The candidate's fitted values are

$$\widehat{u}_{ik}^{(j,\text{lin})} = \widehat{a}_k + \widehat{b}_{jk} x_{ij}^*.$$

This direct calculation avoids fitting a separate regression model through `lm()` for every covariate and update.

For a P-spline candidate, the basis matrix $\mathbf{B}_j$, penalty matrix $\mathbf{K}_j$ and penalty value λ_j have already been prepared for the covariate. The coefficient vector is obtained from

$$\widehat{\gamma}_{jk} = (\mathbf{B}_j^\top \mathbf{B}_j + \lambda_j \mathbf{K}_j)^{-1} \mathbf{B}_j^\top \mathbf{u}_k,$$

which gives the fitted gradient

$$\widehat{\mathbf{u}}_k^{(j,\text{spl})} = \mathbf{B}_j \widehat{\gamma}_{jk}.$$

The basis and penalty matrices remain fixed throughout boosting. Only the right-hand side and the resulting coefficients change with the working gradient.

The argument `learner` determines which candidates enter the comparison. The setting `"linear"` considers one linear candidate per covariate, `"spline"` considers one P-spline candidate per covariate and `"auto"` considers both. The selected candidate is

$$(j_k^*, \ell_k^*) = \arg \min_{j, \ell} \sum_{i=1}^n \left(u_{ik} - \widehat{u}_{ik}^{(j, \ell)} \right)^2,$$

where ℓ denotes an allowed learner type. For P-spline candidates, this comparison uses the ordinary residual sum of squares rather than the penalized objective used to estimate the B-spline coefficients. The fitted values of the selected candidate are multiplied by the parameter-specific step size and added to the current predictor:

$$\eta_{ik} \leftarrow \eta_{ik} + \nu_k \widehat{u}_{ik}^{(j_k^*, \ell_k^*)}, \quad \theta_{ik} \leftarrow g_k^{-1}(\eta_{ik}).$$

The package records the selected parameter, covariate, learner type and update so that the fitted path can later be reconstructed.

The cyclic and noncyclic schedules from Section 2.3 organize these updates differently. Under cyclic updating, the engine visits every active parameter in the order stored by the family object. Its gradient is recomputed immediately before its update, so later parameter updates in the same round use the estimates produced by earlier ones. A parameter is skipped after reaching its own stopping budget. One cyclic round can therefore contain as many applied updates as there are active parameters.

Under noncyclic updating, the engine first constructs the best candidate for each parameter from the same current model. It then calculates the total risk that would result from applying each candidate and commits the candidate with the smallest resulting risk. All other candidates are discarded and recalculated in the next round. A noncyclic round always contains exactly one applied update and the model uses one shared stopping budget. Ties are resolved in favor of the parameter that appears first in the family object's parameter order. The implementation does not separately require the selected candidate to improve on the current risk. If every candidate increases the risk, the one with the smallest resulting risk is still applied.

The package stores the risk before boosting and after every outer round. The meaning of one recorded round differs between the two schedules: a cyclic round may contain several parameter updates while a noncyclic round contains exactly one. Risk traces indexed by round should therefore not be interpreted as comparisons after the same number of applied updates.

When `patience` is supplied, the package additionally tracks risk on one validation sample. The training data are standardized using their own means and standard deviations and the same transformation is applied to the validation data. The search stops after the specified number of consecutive rounds without an improvement in validation risk. The selected number of rounds is then used to refit the model on the full data. Section 3.3 distinguishes this internal validation procedure from cross-validation.

3.3 Tuning and package interface

For the Gaussian location-scale model, `asp26boost` selects a common stopping iteration by V -fold cross-validation. The function `cv_boost_gaussian()` receives a maximum

iteration budget M , fixed learning rates, an update schedule and a base-learner mode. It divides the observations into V folds of approximately equal size. Users can reproduce the random assignment through the `seed` argument or supply their own assignment through `folds`.

For each fold v , the model is fitted once on the remaining observations up to iteration M . Earlier fits can be reconstructed from the stored boosting path, so evaluating every $m \leq M$ does not require a separate model fit. Let $\mathcal{I}_v$ denote the observations in fold v and let $n_v = |\mathcal{I}_v|$. The held-out risk at iteration m is

$$\hat{\rho}_v(m) = \frac{1}{n_v} \sum_{i \in \mathcal{I}_v} \left[\log \hat{\sigma}_i^{(-v,m)} + \frac{\left(y_i - \hat{\mu}_i^{(-v,m)} \right)^2}{2 \left(\hat{\sigma}_i^{(-v,m)} \right)^2} \right], \quad (6)$$

where $\hat{\mu}_i^{(-v,m)}$ and $\hat{\sigma}_i^{(-v,m)}$ are obtained from the model fitted without fold v , evaluated after m iterations. Equation (6) is the mean Gaussian negative log-likelihood without the constant $\frac{1}{2} \log(2\pi)$. Omitting this constant does not affect the selected model.

The fold risks are averaged with equal weight,

$$\bar{\rho}(m) = \frac{1}{V} \sum_{v=1}^V \hat{\rho}_v(m), \quad \hat{m}_{\text{stop}} = \arg \min_{m \in \mathcal{M}} \bar{\rho}(m), \quad (7)$$

where $\mathcal{M}$ contains the iterations evaluated in every fold. Without evaluation stopping, $\mathcal{M} = \{1, \dots, M\}$. If `patience_cv` is supplied, the risk evaluation for a fold ends after the stated number of iterations without improvement. Only iterations reached by every fold remain eligible. This option reduces the number of prediction steps, although each fold model has already been fitted up to M . Selecting the iteration with the lowest cross-validated predictive risk follows the standard cross-validation principle and its use for boosted GAMLSS models [Stone, 1974, Mayr et al., 2012a].

After selecting $\hat{m}_{\text{stop}}$, the function refits the model on the complete data set. The returned object contains the selected iteration, the averaged risk path, the fold-specific risks and the final model. This is a one-dimensional search: both Gaussian predictors receive the same stopping iteration even though their learning rates may differ.

The function `cv_boost_grid()` repeats this procedure over combinations of `nu_mu`, `nu_sigma` and the base-learner mode. All configurations use the same fold assignment, while the update schedule remains fixed. The function can also receive several maximum

iteration budgets. Since the path for a smaller budget is already contained in a fit with a larger budget, using several budgets for an otherwise unchanged configuration repeats the same calculation. A single sufficiently large budget is therefore enough.

Fitted models provide `print()`, `predict()`, `summary()` and `plot()` methods. The Gaussian model has its own methods, while the Gamma, Poisson and Binomial wrappers share the methods defined for the `boost_dist` class. Predictions can be reconstructed at an earlier iteration and calculated for new observations using the transformations stored during fitting. The plotting methods display coefficient paths, selection frequencies, empirical-risk paths and fitted partial effects.

Randomness enters the package when folds or internal validation samples are generated. Both procedures accept a seed. Recording the seed together with the R and package versions makes the resampling design reproducible, although numerical results may still differ across software versions or platforms.

4 Simulation Study

4.1 Study design

The simulation study is structured following the ADEMP framework proposed by [Morris et al. \[2019\]](#), which organises every simulation study around five components: *Aims*, *Data-generating mechanisms*, *Estimands*, *Methods*, and *Performance measures*. We conduct three separate simulation experiments, each targeting a distinct aspect of `asp26boost`: its ability to perform covariate selection in high-dimensional settings (Experiment 1), its computational and algorithmic efficiency relative to an established reference implementation (Experiment 2), and its ability to recover non-linear relationships via B-Spline base learners (Experiment 3). Together, the three experiments assess the statistical regularization properties, computational implementation, and modeling flexibility of `asp26boost`. All simulations are conducted using the R package `simsalapar` [[Hofert and Mächler, 2016](#)], which handles the execution grid, random-number seeding, error catching, and runtime measurement. To ensure full reproducibility, all code, seeds, generated data, fitted models, and summarized results are provided in [Appendix C](#).

Table [1](#) provides a structured overview of all three experiments.

Table 1: Overview of the three simulation experiments. All experiments use $n = 250$ observations for model fitting and N_{sim} repetitions. Experiment 3 additionally uses $n_{\text{test}} = 100$ independently generated observations for out-of-sample evaluation. NLL = negative log-likelihood.

	Experiment 1 Covariate Selection	Experiment 2 Computational Efficiency	Experiment 3 Non-linear Effects
Aim	Assess whether component-wise boosting with early stopping effectively distinguishes signal from noise covariates and provides implicit regularisation.	Quantify the computational efficiency and algorithmic behaviour of <code>asp26boost</code> relative to <code>gamboostLSS</code> .	Assess whether B-Spline base learners substantially improve the recovery of non-linear location and scale effects compared with linear base learners.
Covariates	$p \in \{50, 100, 1000\}$; 3 signal vars. per submodel, remainder noise	$p = 20$; 10 signal vars. per submodel	$p = 10$; 5 signal vars. per submodel
DGM	Linear Gaussian location-scale model; $\mathbf{Z}_i = \mathbf{X}_i$	Same linear structure as Exp. 1 with fixed $p = 20$	Non-linear Gaussian location-scale model; $\mathbf{Z}_i = \mathbf{X}_i$
Estimand	Selection frequency of signal and noise covariates, separately for μ and $\log \sigma$; secondary: in-sample NLL	Number of boosting iterations until early stopping and total wall-clock runtime; secondary: in-sample NLL	Out-of-sample NLL on a held-out test set
Methods	<code>asp26boost</code> vs. <code>lmls</code>	<code>asp26boost</code> (cyclical) vs. <code>gamboostLSS</code> (cyclical)	<code>asp26boost</code> with <code>learner = "spline"</code> vs. <code>learner = "linear"</code>
Performance	Signal and noise selection frequencies and their differences between <code>asp26boost</code> and <code>lmls</code>	Mean runtime and iteration count, together with absolute and relative differences	Difference in out-of-sample NLL between spline and linear learners

4.1.1 Aims

Covariate selection. Assess whether component-wise gradient boosting with early stopping constitutes an effective implicit variable-selection procedure, even when the number of candidate covariates p is large relative to the number of true signals.

Computational efficiency. Evaluate how competitive the pure R implementation of `asp26boost` is relative to `gamboostLSS` [Hofner et al., 2016].

Non-linear effects. Assess whether the B-Spline base learner, implemented in `asp26boost`, substantially improves the recovery of non-linear relationships in both the location and scale parts of the model compared with a linear base learner.

4.1.2 Data-generating mechanisms

All three experiments share the same basic observation model. For each simulation repetition we draw $n = 250$ observations. Preliminary testing revealed that `lmls` exhibited numerical instability already at $n = 150$ for $p = 50$, as the high-dimensional design matrix caused convergence failures in the non-regularised estimator. Increasing the sample size to $n = 250$ resolved this issue while remaining consistent with settings reported by Hofner et al. 2016. For each observation $i = 1, \dots, n$ we generate a covariate vector

$$\mathbf{X}_i = (X_{i1}, \dots, X_{ip})^\top, \quad X_{ij} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1), \quad (8)$$

and set the scale-submodel design matrix equal to the location-submodel design matrix, $\mathbf{Z}_i = \mathbf{X}_i$, so that the same covariates are in principle available to both submodels. Throughout all three experiments, the covariate vectors are drawn independently, so that the covariance matrix is the identity: $\Sigma = I_p$. This choice was deliberate since the focus of the study lies on the regularisation, efficiency, and non-linear recovery properties of the boosting procedure itself. Introducing collinearity would complicate interpretation without adding insight into the core research questions. However, this remains an additional dimension worth exploring in the future.

The response is drawn from a Gaussian location-scale model,

$$Y_i \mid \mathbf{X}_i \sim \mathcal{N}(\mu_i, \sigma_i^2), \quad (9)$$

where the exact specification of μ_i and σ_i differs across experiments as described below.

Experiment 1: Covariate selection. The true data-generating model contains three signal variables for the location submodel and three distinct signal variables for the scale submodel. The remaining $p - 6$ variables are pure noise variables for both submodels. The location linear predictor is

$$\mu_i = 2X_{i1} - 1.5X_{i2} + X_{i3}, \quad \boldsymbol{\beta}_\mu = (2, -1.5, 1, 0, \dots, 0)^\top, \quad (10)$$

and the scale linear predictor is specified on the $\log \sigma$ scale as

$$\log(\sigma_i) = 0.5Z_{i4} - 0.4Z_{i5} + 0.3Z_{i6}, \quad \boldsymbol{\beta}_\sigma = (0, 0, 0, 0.5, -0.4, 0.3, 0, \dots, 0)^\top. \quad (11)$$

Thus X_1, X_2, X_3 are the signal variables for μ , while X_4, X_5, X_6 are the signal variables for $\log \sigma$. The non-overlapping signal structure means that each submodel has its own set of true predictors, while the remaining variables are noise. We repeat this experiment three times increasing the total number of covariates $p \in \{50, 100, 1000\}$, yielding 47, 97, and 997 noise variables, respectively.

Experiment 2: Computational efficiency. We use the same linear location-scale structure as in Experiment 1 but fix $p = 20$. The first ten covariates serve as signal variables for μ and the remaining ten as signal variables for $\log \sigma$:

$$\mu_i = \mathbf{X}_i \boldsymbol{\beta}_\mu, \quad (12)$$

with

$$\boldsymbol{\beta}_\mu = (1.0, -0.8, 0.6, -0.5, 0.4, -0.3, 0.25, -0.2, 0.15, -0.1, 0, \dots, 0)^\top, \quad (13)$$

and

$$\log(\sigma_i) = \mathbf{X}_i \boldsymbol{\beta}_\sigma, \quad (14)$$

with

$$\boldsymbol{\beta}_\sigma = (0, \dots, 0, 0.5, -0.4, 0.35, -0.3, 0.25, -0.2, 0.15, -0.1, 0.08, -0.06)^\top. \quad (15)$$

Experiment 3: Non-linear effect recovery. We replace the linear signal functions with the following non-linear specifications:

$$\mu_i = \sin(2X_{i1}) + X_{i2}^2 - 0.5 \cos(\pi X_{i3}) + 0.8 X_{i4} - 0.6 X_{i5}, \quad (16)$$

$$\log(\sigma_i) = 0.5 \sin(\pi X_{i6}) + 0.4(X_{i7}^2 - 1) - 0.3 X_{i8} + 0.2 X_{i9} - 0.8 \sqrt{|X_{i10}|}. \quad (17)$$

The location signal combines smooth periodic terms ($\sin$, $\cos$), a quadratic effect, and linear effects. The scale signal contains both non-linear and linear effects. These functional forms were chosen to assess whether B-Spline base learners substantially improve the recovery of genuinely non-linear effects. Covariates $X_1, \dots, X_5$ affect the location model, while $X_6, \dots, X_{10}$ affect the scale model. We fix $p = 10$.

4.1.3 Estimands

Experiment 1. The primary estimands are the *selection frequencies of signal and noise covariates*, computed separately for the location and scale submodels. For each true signal variable, we record, across all N_{sim} repetitions, the proportion of runs in which the corresponding estimated coefficient is non-zero at the end of boosting. Analogously, for each noise variable, we record the proportion of runs in which its estimated coefficient is non-zero. As a secondary estimand we report the in-sample negative log-likelihood of the fitted model as a gauge for model quality.

Experiment 2. The primary estimands are (i) the *number of boosting iterations* until the internal early-stopping criterion is triggered and (ii) the *total wall-clock runtime* for a model fit. The two quantities capture complementary aspects of computational behaviour: runtime primarily measures the computational cost of the implementation, whereas the number of iterations reflects the behaviour of the boosting algorithm and its update scheme. The secondary estimand is the in-sample negative log-likelihood, which allows us to check that differences in computational behaviour are not accompanied by systematic differences in model quality.

Experiment 3. The sole estimand is the *out-of-sample negative log-likelihood* evaluated on an independently generated held-out test set. For each simulation repetition, $n_{\text{test}} = 100$ additional observations are generated independently from the same data-

generating mechanism and are not used for model fitting. The same test set is used to evaluate both competing models within each repetition, yielding a paired comparison of their out-of-sample performance.

4.1.4 Methods

Each experiment is repeated $N_{\text{sim}} = 1,000$ times.

Experiment 1. In each repetition we fit (a) `asp26boost` with linear base learners and early stopping and (b) `lmls` [Riebl, 2024] as a baseline for comparison. `lmls` is a non-regularised Gaussian location-scale regression package that is the parent package to `asp26boost`.

Experiment 2. We fit two boosting models: one using `asp26boost` and the other using the `gamboostLSS` package [Hofner et al., 2016]. Both models use a cyclical update scheme and the same learning rates.

Because `gamboostLSS` does not provide a built-in early-stopping mechanism, we implemented the same patience-based early-stopping procedure used in `asp26boost` for it, so that both methods operate under equivalent stopping conditions.

To obtain runtime estimates that are directly comparable between methods, both `asp26boost` and `gamboostLSS` are timed on a pure model fit: after the optimal number of iterations has been determined by the early-stopping procedure, each model is refitted once on the full dataset using that fixed iteration count, without any early-stopping overhead. The runtime of this final refit is the quantity compared across methods.

Experiment 3. We fit `asp26boost` twice in each repetition: once with `learner = "spline"` (B-Spline base learners) and once with `learner = "linear"` (ordinary linear base learners). The same stopping criterion, maximum number of iterations, and learning rates are used in both variants. Both models are fitted to the same training data and evaluated on the same independently generated test set.

4.1.5 Performance measures

Experiment 1. The main performance measures are the *mean signal-covariate selection frequency* and the *mean noise-covariate selection frequency*, reported separately for the location and scale submodels and for each level of p . The difference in these quantities between `asp26boost` and `lmls` is used to quantify the effect of implicit

regularisation.

Experiment 2. We compare the *mean wall-clock runtime* of the pure model fit for each method, together with the *mean number of boosting iterations* until early stopping is triggered.

Experiment 3. The primary performance measure is the *mean out-of-sample negative log-likelihood*. A lower negative log-likelihood indicates a better out-of-sample fit. Because the spline and linear models are evaluated on the same test observations within each simulation repetition, their performance can be compared directly across repetitions.

In addition to the key performance metrics the distribution and summarizing quantities of all the estimands are provided.

A note on model comparison criteria. Standard Information Criteria like AIC are not directly applicable in its usual parameter-counting form to gradient boosting models, because the effective complexity of a regularised, algorithmic estimator is not adequately characterised by simply counting the estimated coefficients [Hofner et al., 2016]. Following the recommendation of Hofner et al. [2016], we therefore use the empirical risk—the (joint Gaussian) negative log-likelihood—as our model comparison criterion throughout. For a Gaussian location-scale model there is a single joint log-likelihood that depends simultaneously on both μ and σ :

$$-\ell(\boldsymbol{\mu}, \boldsymbol{\sigma}; \mathbf{y}) \propto \sum_{i=1}^n \left[\log(\sigma_i) + \frac{(y_i - \mu_i)^2}{2\sigma_i^2} \right]. \quad (18)$$

For Experiments 1 and 2, where variable selection and computational behaviour are the primary concerns, we use the in-sample negative log-likelihood as a secondary quality check. For Experiment 3, where the central question is generalisation of non-linear fits, we use the out-of-sample negative log-likelihood evaluated on the held-out test set.

4.2 Results

4.2.1 Experiment 1: Covariate selection

The results confirm that component-wise boosting with early stopping constitutes an effective implicit variable-selection procedure. Across all three dimensionalities, **asp26boost** consistently identifies the true signal covariates while suppressing the majority of noise variables, and does so without a single estimation failure - even at $p = 1,000$, where the number of covariates is four times larger than the sample size.

Signal selection is stronger in the location submodel than in the scale submodel across all values of p . This asymmetry is consistent with the coefficient magnitudes in the data-generating mechanism: the scale effects are smaller in absolute value and thus harder to distinguish from noise. Both signal and noise selection frequencies decline as p increases, reflecting the deteriorating signal-to-noise ratio as the covariate space grows.

The **lmls** baseline, by contrast, selects all available covariates, given the model was fitted successfully, at $p = 50$ and fails entirely for $p \in \{100, 1000\}$, producing no result in any repetitions at those dimensionalities. This underscores that the variable-selection behaviour of **asp26boost** is not a trivial feature but a practically important property of the boosting approach. Consequently, we did not only show that **asp26boost** is capable of variable selection, but went beyond our aim and demonstrated that **asp26boost** still produces meaningful estimates where other methods already fail.

Regarding in-sample fit, **lmls** achieves a lower in-sample NLL at $p = 50$, but this reflects overfitting rather than superior model quality. More informative is the behaviour of **asp26boost** itself: the NLL distribution widens considerably as p grows, consistent with the decrease in signal selection frequency.

Figure 2 shows the mean selection frequencies and Figure 3 shows the distributions of the NLL. Full numerical summaries are provided in Tables 2 and 3 in the Appendix.

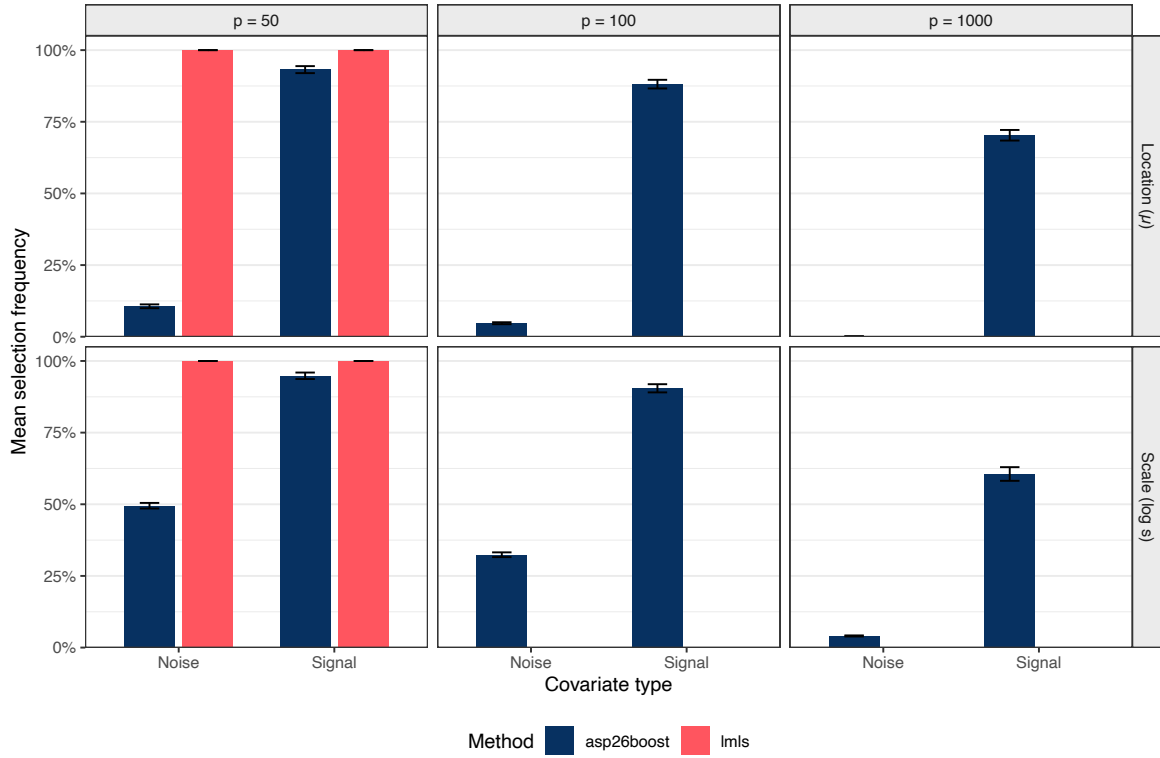


Figure 2: Mean selection frequency of signal and noise covariates for `asp26boost`, by submodel and number of covariates p . Error bars indicate 95% Monte Carlo confidence intervals. `lmls` produces no estimation result for $p \in \{100, 1000\}$.

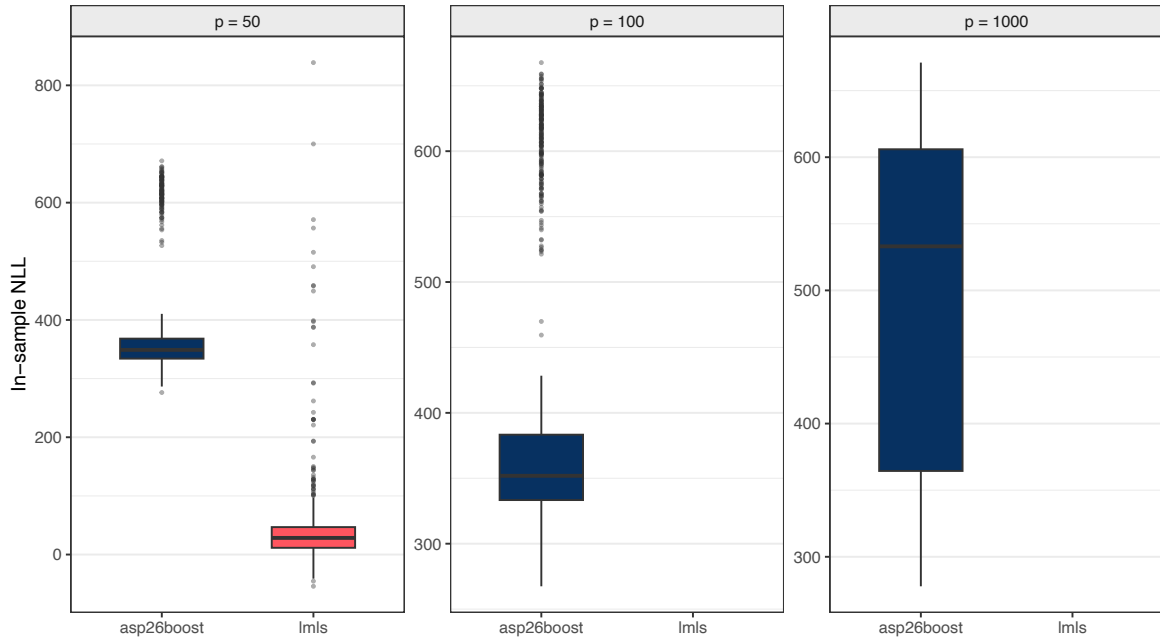
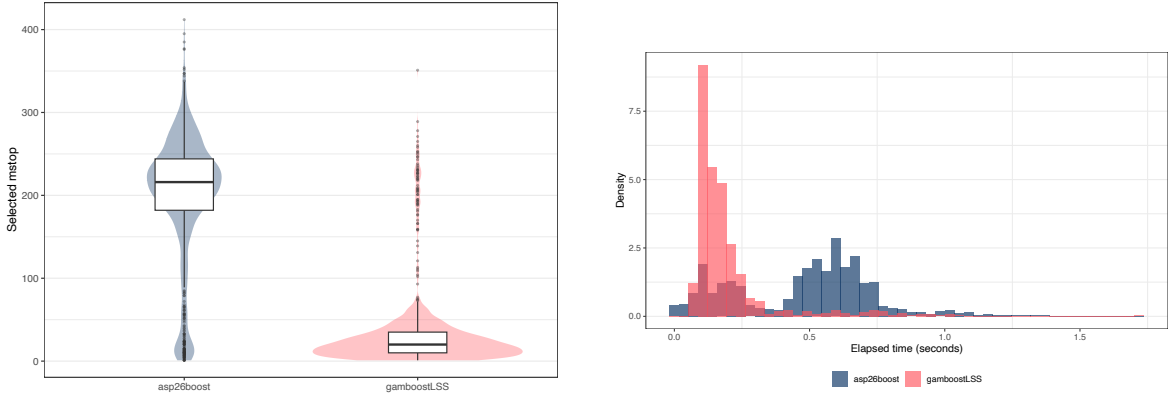


Figure 3: In-sample NLL distributions for `asp26boost` and `lmls`, faceted by p . `lmls` produces no estimation result for $p \in \{100, 1000\}$.

4.2.2 Experiment 2: Computational efficiency

`asp26boost` is slower than `gamboostLSS` both in terms of wall-clock runtime and the number of iterations required to trigger early stopping, which is unsurprising given that `gamboostLSS` is built on a compiled C backend while `asp26boost` is a pure R implementation. The runtime increases by a factor of 2.5 (mean 479 ms vs. 194 ms).

What is more interesting is that `gamboostLSS` also converges in far fewer iterations (mean 34.1 vs. 199.6 for `asp26boost`) despite using the same nominal learning rate, suggesting that the runtime gap is not purely an implementation artefact but partly reflects a genuine difference in algorithmic behaviour. Figure 4a shows the iteration count distributions; Figure 4b shows the runtime distributions. Full summary statistics are given in Table 4 in the Appendix.



(a) Distribution of the number of boosting iterations selected by the early-stopping procedure across $N_{\text{sim}} = 1,000$ repetitions.

(b) Distribution of pure model-fitting wall-clock runtime (seconds), excluding early-stopping search overhead.

Figure 4: Computational efficiency results for Experiment 2.

4.2.3 Experiment 3: Non-linear Effect Recovery

The B-Spline base learner in `asp26boost` substantially outperforms the linear base learner when the true data-generating mechanism contains non-linear effects. The spline learner achieves a lower out-of-sample NLL in 98% of all simulation repetitions, and the magnitude of the improvement is practically relevant: the median out-of-sample NLL is approximately halved (from 140.1 to 64.7), a reduction of around 54%. We report medians rather than means, as both learners produce influential outliers skewing the mean. Figure 5 shows the distribution of the paired NLL difference; full summaries are given in Table 5 in the Appendix.

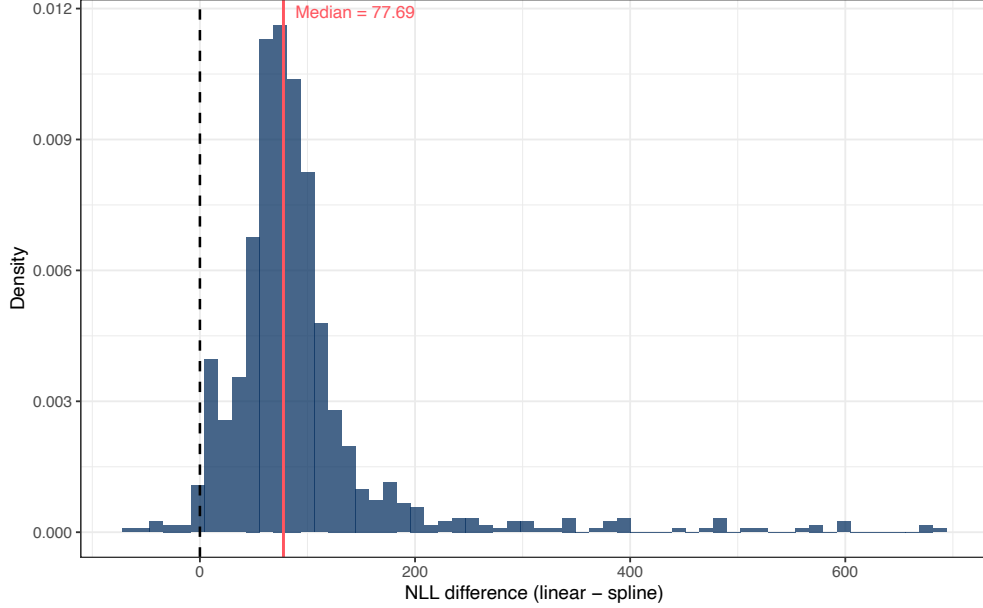


Figure 5: Distribution of the paired out-of-sample NLL difference (linear – spline); truncated at 2.5% and 97.5% quantiles. Positive values indicate that the spline learner achieves a lower out-of-sample NLL.

5 Conclusion

In this report, we present **asp26boost**, an R implementation of component-wise gradient boosting for distributional regression. We describe the statistical foundations and implementation of the method, including parameter-specific predictors, different base learners, cyclic and noncyclic updating, and early stopping. We further present the structure of the package and demonstrate its application and effectiveness in a reproducible simulation study.

Our simulation study shows that **asp26boost** provides effective variable selection in high-dimensional settings and is able to capture nonlinear relationships when using spline base learners. At the same time, the comparison with **gamboostLSS** highlights differences in computational efficiency and convergence behaviour between the two implementations.

Several directions for future work remain. In particular, the current cross-validation procedure selects a common stopping iteration for the location and scale parameters, although these parameters may require different degrees of regularization. Extending the stopping-iteration search to both parameters independently would therefore be a natural improvement. Furthermore, the substantially different convergence behaviour of **asp26boost** and **gamboostLSS** warrants further investigation to determine whether it is

primarily caused by implementation details, or algorithmic differences. The performance of the implemented cross-validation procedure itself should also be evaluated more systematically, for example against alternative tuning strategies. Finally, the simulation study assumes independent covariates. Investigating how correlation and stronger covariance structures affect variable selection and model estimation would provide a more comprehensive assessment of the method.

Overall, `asp26boost` provides a flexible framework for component-wise boosting in distributional regression and establishes a foundation for further methodological and computational extensions.

References

- Peter Bühlmann and Torsten Hothorn. Boosting algorithms: Regularization, prediction and model fitting. *Statistical Science*, 22(4):477–505, 2007. doi: 10.1214/07-STS242.
- Peter Bühlmann and Bin Yu. Boosting with the L_2 loss: Regression and classification. *Journal of the American Statistical Association*, 98(462):324–339, 2003. doi: 10.1198/016214503000125.
- Paul H. C. Eilers and Brian D. Marx. Flexible smoothing with B-splines and penalties. *Statistical Science*, 11(2):89–121, 1996. doi: 10.1214/ss/1038425655.
- Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.
- Jim Hester. *covr: Test Coverage for Packages*, 2023. URL <https://CRAN.R-project.org/package=covr>. R package version 3.6.4.
- Marius Hofert and Martin Mächler. Parallel and Other Simulations in *R* Made Easy: An End-to-End Study. *Journal of Statistical Software*, 69(4), 2016. ISSN 1548-7660. doi: 10.18637/jss.v069.i04. URL <http://www.jstatsoft.org/v69/i04/>.
- Benjamin Hofner, Andreas Mayr, and Matthias Schmid. **gamboostLSS** : An *R* Package for Model Building and Variable Selection in the GAMLSS Framework. *Journal of Statistical Software*, 74(1), 2016. ISSN 1548-7660. doi: 10.18637/jss.v074.i01. URL <http://www.jstatsoft.org/v74/i01/>.
- Torsten Hothorn, Peter Bühlmann, Thomas Kneib, Matthias Schmid, and Benjamin Hofner. Model-based boosting 2.0. *Journal of Machine Learning Research*, 11(71): 2109–2113, 2010. URL <https://www.jmlr.org/papers/v11/hothorn10a.html>.
- Andreas Mayr, Nora Fenske, Benjamin Hofner, Thomas Kneib, and Matthias Schmid. Generalized additive models for location, scale and shape for high dimensional data—a flexible approach based on boosting. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 61(3):403–427, 2012a. doi: 10.1111/j.1467-9876.2011.01033.x.
- Andreas Mayr, Benjamin Hofner, and Matthias Schmid. The importance of knowing

- when to stop: A sequential stopping rule for component-wise gradient boosting. *Methods of Information in Medicine*, 51(2):178–186, 2012b. doi: 10.3414/ME11-02-0030.
- Luca Messerschmidt, Bjarne Herbst, and Benedict Schnitzler. *asp26boost: Component-wise Gradient Boosting for GAMLSS-type Distributional Regression*, 2026. URL <https://gitlab.gwdg.de/bjarne.herbst01/asp26boost.git>. R package version 0.1.0.
- Tim P. Morris, Ian R. White, and Michael J. Crowther. Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, 38(11):2074–2102, May 2019. ISSN 0277-6715, 1097-0258. doi: 10.1002/sim.8086. URL <https://onlinelibrary.wiley.com/doi/10.1002/sim.8086>.
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2024. URL <https://www.R-project.org/>.
- Hannes Riebl. *lmls: Gaussian Location-Scale Regression*, 2024. URL <https://hriebl.github.io/lmls/>. R package version 0.1.1, <https://github.com/hriebl/lmls>.
- R. A. Rigby and D. M. Stasinopoulos. Generalized additive models for location, scale and shape. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 54(3):507–554, 2005. doi: 10.1111/j.1467-9876.2005.00510.x.
- Matthias Schmid and Torsten Hothorn. Boosting additive models using component-wise P-splines. *Computational Statistics & Data Analysis*, 53(2):298–311, 2008. doi: 10.1016/j.csda.2008.09.009.
- D. Mikis Stasinopoulos and Robert A. Rigby. Generalized additive models for location scale and shape (GAMLSS) in R. *Journal of Statistical Software*, 23(7):1–46, 2007. doi: 10.18637/jss.v023.i07.
- M. Stone. Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(2):111–147, 1974. doi: 10.1111/j.2517-6161.1974.tb00994.x.
- Janeke Thomas, Andreas Mayr, Bernd Bischl, Matthias Schmid, Adam Smith, and Benjamin Hofner. Gradient boosting for distributional regression: Faster tuning and

improved variable selection via noncyclical updates. *Statistics and Computing*, 28(3): 673–687, 2018. doi: 10.1007/s11222-017-9754-6.

A Additional figures and tables

Table 2: Experiment 1: Mean selection frequency of signal and noise covariates, by method, submodel, and number of covariates p . MC-SE: Monte Carlo standard error. `lmls` produces no estimates for $p \in \{100, 1000\}$.

p	Method	Submodel	Type	Mean freq.	MC-SE
50	<code>asp26boost</code>	μ	Signal	0.9320	0.0062
			Noise	0.1062	0.0034
		$\log \sigma$	Signal	0.9483	0.0058
			Noise	0.4950	0.0049
	<code>lmls</code>	μ	Signal	1.0000	0.0000
			Noise	1.0000	0.0000
		$\log \sigma$	Signal	1.0000	0.0000
			Noise	1.0000	0.0000
100	<code>asp26boost</code>	μ	Signal	0.8813	0.0077
			Noise	0.0472	0.0017
		$\log \sigma$	Signal	0.9047	0.0073
			Noise	0.3240	0.0042
	<code>lmls</code>	complete failure (1,000/1,000 reps)		—	
1000	<code>asp26boost</code>	μ	Signal	0.7030	0.0094
			Noise	0.0015	0.0001
		$\log \sigma$	Signal	0.6057	0.0122
			Noise	0.0406	0.0010
	<code>lmls</code>	complete failure (1,000/1,000 reps)		—	

Table 3: Experiment 1: In-sample NLL summary statistics by method and p . *NAs*: number of failed fits.

p	Method	Min	Q1	Median	Mean	Q3	Max	NAs
50	<code>asp26boost</code>	276.3	334.0	349.0	375.6	368.1	671.2	0
	<code>lmls</code>	−54.1	11.5	28.4	38.4	46.8	838.8	15
100	<code>asp26boost</code>	267.4	333.4	351.9	399.5	383.3	667.7	0
	<code>lmls</code>	complete failure						
1000	<code>asp26boost</code>	277.8	364.5	533.1	493.9	605.9	671.0	0
	<code>lmls</code>	complete failure						

Table 4: Experiment 2: Summary statistics for wall-clock runtime (seconds), selected iteration count, and in-sample NLL, by method.

Estimand	Method	Min	Q1	Median	Mean	Q3	Max	NAs
Runtime (s)	asp26boost	0.00	0.23	0.53	0.48	0.64	1.38	0
	gamboostLSS	0.07	0.11	0.14	0.19	0.20	1.72	0
Iterations	asp26boost	1	182	216	199.6	244	412	0
	gamboostLSS	1	10	20	34.1	35	351	0
In-sample NLL	asp26boost	294.1	338.9	356.8	378.3	384.3	632.3	0
	gamboostLSS	55.3	279.7	300.1	290.8	320.2	409.7	0

Table 5: Experiment 3: Out-of-sample NLL summary by base learner and paired difference (linear – spline).

	Min	Q1	Median	Mean	Q3	Max	NAs	Spline wins
Linear	96.6	125.2	140.1	8,846.1	168.2	8,422,653.7	1	—
Spline	0.7	47.6	64.7	500.8	93.2	376,744.8	0	—
Difference	−2186.7	55.0	77.7	8,344.9	105.5	8,045,909.0	1	98%

B Convergence plots

Figure 6 shows the total-risk trajectory referenced in Section 3.2, for the same simulated data as the vignette (Gaussian location-scale model, $n = 1000$, 10 covariates, 3 informative per submodel), fit with $m_{\text{stop}} = 300$ and $\nu_{\mu} = \nu_{\sigma} = 0.05$ under both update schedules.

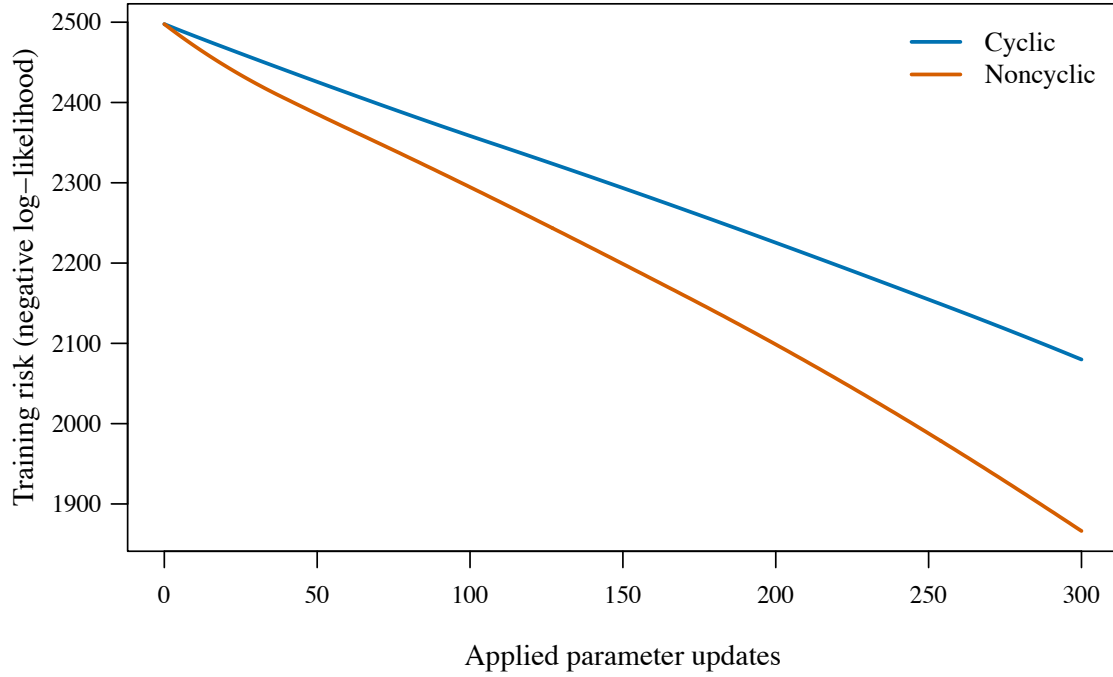


Figure 6: Total training risk (negative log-likelihood) by boosting round, under the cyclic and noncyclic update schedules, on identical simulated data and hyperparameters. Both schedules trace out a monotonically decreasing risk path, as described in Section 3.2; the noncyclic schedule reaches a higher final risk at the same m_{stop} here because its shared iteration budget is split across both parameters rather than granting each its own.

C Reproducibility

The complete code and data used to conduct the simulation study are available from the authors upon request.

This contains the data-generating processes and simulation setups, scripts for running the individual simulation experiments, and the code used to process and visualize the resulting simulation output. The main simulation configuration and experimental settings are collected in the `sim` directory. Generated simulation results are stored separately. The resulting figures and tables in this work can therefore be reproduced directly from the provided code and simulation output.

A more detailed description is provided in the `README.md` located at the root.

D Package manual

The following pages reproduce **asp26boost**'s reference manual, generated directly from the package's documentation via `R CMD Rd2pdf`, so that every exported function's arguments, return value and examples are available without installing the package.

Package ‘asp26boost’

September 3, 2026

Type Package

Title Gradient Boosting

Version 0.1.0

Description Implements a component-wise gradient boosting algorithm for Gaussian location-scale models, as well as Gamma, Poisson, and Binomial response families, with linear or P-spline base learners, prediction, plotting, printing, and summary methods.

License MIT + file LICENSE

Encoding UTF-8

LazyData true

RoxygenNote 7.3.3

Imports stats,
graphics,
splines

Suggests testthat (>= 3.0.0),
knitr,
rmarkdown

Config/testthat/edition 3

VignetteBuilder knitr

R topics documented:

boost_binomial	2
boost_gamma	3
boost_gaussian	5
boost_poisson	7
cv_boost_gaussian	9
cv_boost_grid	10
generate_folds	11
plot.boost_dist	11
plot.boost_gaussian	12
plot.cv_boost_gaussian	14

predict.boost_dist	14
predict.boost_gaussian	15
predict.cv_boost_gaussian	17
predict.cv_boost_grid	18
print.boost_dist	18
print.boost_gaussian	19
print.cv_boost_gaussian	19
print.cv_boost_grid	20
print.summary.boost_dist	20
print.summary.boost_gaussian	21
print.summary.cv_boost_gaussian	21
print.summary.cv_boost_grid	22
summary.boost_dist	22
summary.boost_gaussian	23
summary.cv_boost_gaussian	24
summary.cv_boost_grid	24

Index	25
--------------	-----------

boost_binomial	<i>Binomial/Bernoulli Gradient Boosting</i>
----------------	---

Description

Fits a component-wise gradient boosting model for binary responses, estimating $P(y = 1)$ (logit-linked). A single-parameter special case of the same family-driven boosting engine used by [boost_gaussian](#): method is accepted for interface consistency but has no effect, since cyclic and non-cyclic updates coincide when there is only one parameter to boost.

Usage

```
boost_binomial(
  X = NULL,
  y = NULL,
  formula = NULL,
  data = NULL,
  mstop = 100,
  nu = 0.1,
  method = c("cyclic", "noncyclic"),
  patience = NULL,
  validation_split = 0.2,
  seed = NULL,
  learner = c("linear", "spline", "auto")
)
```

Arguments

<code>X</code>	A data frame or matrix of predictors. Ignored if formula is supplied.
<code>y</code>	A numeric response vector; every value must be exactly 0 or 1. Ignored if formula is supplied.
<code>formula</code>	A two-sided formula, e.g. <code>y ~ x1 + x2</code> . When supplied, <code>X</code> and <code>y</code> are constructed from data instead.
<code>data</code>	A data frame containing the variables referenced in formula. Required (and only used) when formula is supplied.
<code>mstop</code>	Number of boosting iterations (single positive integer).
<code>nu</code>	Learning rate.
<code>method</code>	Accepted for interface consistency with <code>boost_gaussian</code> ; has no effect for this single-parameter family.
<code>patience</code>	Optional internal early-stopping patience (see <code>boost_gaussian</code>).
<code>validation_split</code>	Fraction held out for early-stopping search.
<code>seed</code>	Optional seed for the early-stopping train/validation split.
<code>learner</code>	Base learner type: "linear" (default), "spline" (df-equalized P-splines), or "auto" (best of both per column).

Value

A list of class `c("boost_binomial", "boost_dist")`.

Examples

```
set.seed(123)
X <- data.frame(x1 = rnorm(150))
p <- plogis(0.3 + 0.8 * X$x1)
y <- rbinom(150, size = 1, prob = p)

fit <- boost_binomial(X, y, mstop = 100)
```

 boost_gamma

Distributional Gamma Gradient Boosting

Description

Fits a component-wise gradient boosting model for strictly positive, right-skewed responses, jointly estimating the Gamma mean (μ) and shape (ν) parameters (both log-linked). Built on the same family-driven boosting engine as `boost_gaussian`.

Usage

```
boost_gamma(
  X = NULL,
  Z = NULL,
  y = NULL,
  formula = NULL,
  data = NULL,
  mstop = 100,
  nu_mu = 0.1,
  nu_shape = 0.1,
  method = c("cyclic", "noncyclic"),
  patience = NULL,
  validation_split = 0.2,
  seed = NULL,
  learner = c("linear", "spline", "auto")
)
```

Arguments

X	A data frame or matrix of predictors for the mean submodel. Ignored if formula is supplied.
Z	A data frame or matrix of predictors for the shape submodel. Ignored if formula is supplied.
y	A numeric response vector; every value must be strictly positive. Ignored if formula is supplied.
formula	A list with components mu (a two-sided formula) and shape (a one-sided formula). When supplied, X, Z, and y are constructed from data instead.
data	A data frame containing the variables referenced in formula. Required (and only used) when formula is supplied.
mstop	Number of boosting iterations. Either a single positive integer or, for method = "cyclic" only, c(mu = ..., shape = ...).
nu_mu	Learning rate for the mean submodel.
nu_shape	Learning rate for the shape submodel.
method	Boosting update schedule, "cyclic" (default) or "noncyclic" – see boost_gaussian for details.
patience	Optional internal early-stopping patience (see boost_gaussian).
validation_split	Fraction held out for early-stopping search.
seed	Optional seed for the early-stopping train/validation split.
learner	Base learner type: "linear" (default), "spline" (df-equalized P-splines), or "auto" (best of both per column).

Value

A list of class c("boost_gamma", "boost_dist").

Examples

```
set.seed(123)
X <- data.frame(x1 = rnorm(150))
Z <- data.frame(z1 = rnorm(150))
mu <- exp(1 + 0.5 * X$x1)
shape <- exp(0.3 * Z$z1 + 1)
y <- rgamma(150, shape = shape, rate = shape / mu)

fit <- boost_gamma(X, Z, y, mstop = 100)
```

 boost_gaussian

Distributional Gaussian Gradient Boosting

Description

Fits a component-wise gradient boosting model for Gaussian responses, jointly estimating the location (mean) and scale (standard deviation).

Usage

```
boost_gaussian(
  X = NULL,
  Z = NULL,
  y = NULL,
  formula = NULL,
  data = NULL,
  mstop = 100,
  nu_mu = 0.1,
  nu_sigma = 0.1,
  method = c("cyclic", "noncyclic"),
  patience = NULL,
  validation_split = 0.2,
  seed = NULL,
  learner = c("linear", "spline", "auto")
)
```

Arguments

X	A data frame or matrix of predictors for the location (mean) model. Ignored if formula is supplied.
Z	A data frame or matrix of predictors for the scale (log-sd) model. Ignored if formula is supplied.
y	A numeric response vector. Ignored if formula is supplied.
formula	A list with components mu (a two-sided formula, e.g. $y \sim x1 + x2$) and sigma (a formula for the scale submodel, e.g. $\sim z1 + z2$). When supplied, X, Z, and y are constructed from data instead.

data	A data frame containing the variables referenced in formula. Required (and only used) when formula is supplied.
mstop	Number of boosting iterations. Either a single positive integer (applied to both submodels) or, for method = "cyclic" only, a named vector/list c(mu = ..., sigma = ...) giving separate iteration budgets for the two submodels. When patience is supplied, mstop is instead the maximum number of rounds to search (must be a single integer in that case).
nu_mu	Learning rate for the location submodel.
nu_sigma	Learning rate for the scale submodel.
method	Boosting update schedule. "cyclic" (default) alternates location/scale updates each iteration (until each submodel's own mstop budget, if separate, is exhausted). "noncyclic" picks, at each iteration, whichever submodel update yields the larger Gaussian log-likelihood improvement, and updates only that one.
patience	If supplied, enables internal early stopping: a single positive integer giving the number of consecutive rounds without validation-risk improvement allowed before stopping the search early. Requires a single scalar mstop (the maximum rounds to search). The optimal number of rounds found this way (best_round) is then used to refit the model on the <i>full</i> X/Z/y, so the returned model itself is always trained on all the supplied data.
validation_split	Fraction of rows (in (0, 1)) held out to evaluate validation risk during the patience search. Ignored if patience is NULL. Default 0.2.
seed	Optional integer seed for the random train/validation split used when patience is supplied (for reproducibility). Ignored if patience is NULL.
learner	Base learner type for both submodels. "linear" (default) reproduces the original simple-linear-per-column behavior. "spline" fits a df-equalized P-spline per column instead. "auto" lets each boosting step pick whichever of the two gives the lower residual sum of squares.

Details

This implements component-wise gradient boosting for a Gaussian location-scale model (in the spirit of gamboostLSS). The two distribution parameters are updated one base learner at a time:

1. **Location step** – the negative gradient of the Gaussian log-likelihood w.r.t. μ is $u_\mu = (y - \mu)/\sigma^2$. Each column of X is fit by a simple linear base learner ($\text{lm}(u_\mu \sim x_j)$, with intercept); the column with the lowest residual sum of squares is selected and $\hat{\mu}$ is updated by a small step nu_mu.
2. **Scale step** – the negative gradient w.r.t. $\log \sigma$ is $u_\sigma = (y - \mu)^2/\sigma^2 - 1$. A simple linear base learner is fit to each column of Z and the best is selected; $\log \hat{\sigma}$ is updated by a small step nu_sigma. Working on the log scale keeps $\hat{\sigma} > 0$ and naturally bounds the step size.

Under method = "cyclic" both steps are taken every iteration (each submodel stops once it reaches its own mstop budget, while the other keeps going). Under method = "noncyclic" only the step with the larger likelihood improvement is taken each iteration.

Risk trace: the in-sample Gaussian negative log-likelihood is recorded after every round (risk, plus the pre-boosting baseline risk0), so convergence can be inspected directly or via `plot(fit, type = "risk")`.

Internal early stopping: when patience is supplied, a single random train/validation split (`validation_split`) is used to search for the number of rounds (up to `mstop`) that minimizes validation risk, stopping the search once patience rounds pass without improvement. The model that is actually returned is then refit on the complete data with `mstop` set to that optimal round count; the search details (including the validation risk trace) are stored in `fit$early_stopping`.

Predictor standardisation: both X and Z are standardised internally (zero mean, unit variance) before fitting so that all base learners compete on the same scale. Stored coefficients are converted back to the original predictor scale (see below).

Base learners include an intercept: each base learner is a full simple linear regression $\text{lm}(u \sim x_j)$. The intercept term allows the overall level of the linear predictor to be corrected at every step. This matters most for the scale submodel, whose level is initialised at $\log(\text{sd}(y))$ and must be free to move toward the correct residual scale.

Coefficients for both submodels are stored and reported on the *original predictor scale* (not the standardised scale). For the scale submodel the linear predictor is $\log \sigma$, so a coefficient expresses the additive change in $\log \sigma$ per unit change in the original predictor (equivalently, a multiplicative factor $\exp(\cdot)$ on σ).

Value

A list of class "boost_gaussian" containing fitted values, coefficient paths, centering/scaling constants, and diagnostics for both the location and scale submodels.

Examples

```
set.seed(123)
X <- data.frame(x1 = rnorm(100), x2 = rnorm(100))
Z <- data.frame(z1 = rnorm(100), z2 = rnorm(100))
y <- 2 + 3 * X$x1 + exp(0.5 * Z$z1) * rnorm(100)

fit <- boost_gaussian(X, Z, y, mstop = 100, nu_mu = 0.1, nu_sigma = 0.1)
predict(fit, newdata_X = X[1:5, ], newdata_Z = Z[1:5, ])

# Non-cyclic updates: only the more useful submodel update is taken
fit_nc <- boost_gaussian(X, Z, y, mstop = 100, method = "noncyclic")

# Separate iteration budgets for mu and sigma (cyclic only)
fit_2m <- boost_gaussian(X, Z, y, mstop = c(mu = 100, sigma = 30))

# Formula interface
df <- cbind(X, Z, y = y)
fit_f <- boost_gaussian(formula = list(mu = y ~ x1 + x2, sigma = ~ z1 + z2),
                        data = df, mstop = 100)

# Internal early stopping (searches up to mstop rounds on a 80/20 split,
# then refits the chosen number of rounds on all the data)
fit_es <- boost_gaussian(X, Z, y, mstop = 200, patience = 10, seed = 1)
fit_es$early_stopping$best_round
```

boost_poisson	<i>Poisson Gradient Boosting</i>
---------------	----------------------------------

Description

Fits a component-wise gradient boosting model for count responses, estimating the Poisson mean μ (log-linked). A single-parameter special case of the same family-driven boosting engine used by [boost_gaussian](#): method is accepted for interface consistency but has no effect, since cyclic and non-cyclic updates coincide when there is only one parameter to boost.

Usage

```
boost_poisson(
  X = NULL,
  y = NULL,
  formula = NULL,
  data = NULL,
  mstop = 100,
  nu = 0.1,
  method = c("cyclic", "noncyclic"),
  patience = NULL,
  validation_split = 0.2,
  seed = NULL,
  learner = c("linear", "spline", "auto")
)
```

Arguments

X	A data frame or matrix of predictors. Ignored if formula is supplied.
y	A numeric response vector; every value must be non-negative. Ignored if formula is supplied.
formula	A two-sided formula, e.g. $y \sim x_1 + x_2$. When supplied, X and y are constructed from data instead.
data	A data frame containing the variables referenced in formula. Required (and only used) when formula is supplied.
mstop	Number of boosting iterations (single positive integer).
nu	Learning rate.
method	Accepted for interface consistency with boost_gaussian ; has no effect for this single-parameter family.
patience	Optional internal early-stopping patience (see boost_gaussian).
validation_split	Fraction held out for early-stopping search.
seed	Optional seed for the early-stopping train/validation split.
learner	Base learner type: "linear" (default), "spline" (df-equalized P-splines), or "auto" (best of both per column).

Value

A list of class `c("boost_poisson", "boost_dist")`.

Examples

```
set.seed(123)
X <- data.frame(x1 = rnorm(150))
mu <- exp(0.5 + 0.4 * X$x1)
y <- rpois(150, lambda = mu)

fit <- boost_poisson(X, y, mstop = 100)
```

cv_boost_gaussian	<i>k-fold Cross-validation for Hyperparameter Tuning for Boosting Algorithm</i>
-------------------	---

Description

k-fold Cross-validation for Hyperparameter Tuning for Boosting Algorithm

Usage

```
cv_boost_gaussian(
  X,
  Z,
  y,
  k = 5,
  mstop = 100,
  nu_mu = 0.1,
  nu_sigma = 0.1,
  method = "cyclic",
  seed = NULL,
  folds = NULL,
  patience_cv = NULL,
  learner = "linear"
)
```

Arguments

X	data matrix or data frame for location submodel
Z	data matrix or data frame for scale submodel
y	numeric response-vector
k	number of folds
mstop	limit to iterations
nu_mu	learning rate for location submodel

nu_sigma	learning rate for scale submodel
method	boosting update rule ("cyclic", "noncyclic")
seed	random seed for reproducibility
folds	optional fold vector, if random generation is not wanted
patience_cv	threshold for early stopping
learner	base learner type forwarded to boost_gaussian(): "linear" (default), "spline", or "auto"

Value

object of class "cv_boost_gaussian"

cv_boost_grid	<i>CV with Parameter Grid</i>
---------------	-------------------------------

Description

CV with Parameter Grid

Usage

```
cv_boost_grid(
  X,
  Z,
  y,
  k = 5,
  mstop_grid = 200,
  nu_mu_grid = c(0.01, 0.1, 0.3),
  nu_sigma_grid = c(0.01, 0.1, 0.3),
  method = "cyclic",
  learner_grid = "linear",
  seed = NULL
)
```

Arguments

X	data matrix or dataframe for the location submodel
Z	data matrix or dataframe for the scale submodel
y	numeric response vector
k	number of folds
mstop_grid	vector of mstop-values
nu_mu_grid	vector of nu-values for the location submodel
nu_sigma_grid	vector of nu-values for the scale submodel
method	boosting update rule ("cyclic", "noncyclic")
learner_grid	vector of base-learner types to compare, subset of c("linear", "spline", "auto")
seed	optional seed

Value

object of class "cv_boost_grid" containing:

call the matched function call

best_combination data frame row with the best mstop, nu, and learner values

all_results data frame with CV errors for all parameter combinations

final_model fitted boost_gaussian model using the best parameters

generate_folds	<i>Generate CV Folds</i>
----------------	--------------------------

Description

Generate CV Folds

Usage

```
generate_folds(n, k, seed = NULL)
```

Arguments

n	number of observations
k	number of folds
seed	optional seed

Value

integer vector of fold assignments

plot.boost_dist	<i>Plot a Distributional Boosting Model</i>
-----------------	---

Description

Produces diagnostic plots for a fitted boost_gamma, boost_poisson, or boost_binomial object, generalizing [plot.boost_gaussian](#) to any number of distribution parameters.

Usage

```
## S3 method for class 'boost_dist'
plot(
  x,
  type = c("path", "frequency", "risk", "partial"),
  param = NULL,
  top_n = 8,
  scale = c("original", "standardized"),
  variable = NULL,
  ...
)
```

Arguments

x	A fitted object inheriting from class "boost_dist".
type	Plot type: "path", "frequency", "risk", or "partial" – see plot.boost_gaussian for details.
param	Which parameter's submodel to plot (one of x\$parameters); defaults to the first parameter.
top_n	Maximum number of variables shown in the coefficient path plot.
scale	Coefficient scale for type = "path": "original" or "standardized".
variable	Required when type = "partial".
...	Additional graphical arguments passed to base plotting functions.

Value

The input object x, invisibly.

plot.boost_gaussian	<i>Plot a Distributional Gaussian Boosting Model</i>
---------------------	--

Description

Produces diagnostic plots for a fitted boost_gaussian object. Both the location and the scale submodel can be visualised.

Usage

```
## S3 method for class 'boost_gaussian'
plot(
  x,
  type = c("path", "frequency", "risk", "partial"),
  submodel = c("mu", "sigma"),
  top_n = 8,
  scale = c("original", "standardized"),
```

```

    variable = NULL,
    ...
)

```

Arguments

<code>x</code>	A fitted object of class "boost_gaussian".
<code>type</code>	Plot type: "path" for cumulative coefficient paths, "frequency" for variable selection frequencies, "risk" for the in-sample (and, if internal early stopping was used, validation) negative log-likelihood trace over boosting rounds, or "partial" for a variable's fitted partial effect (needed when <code>learner = "spline"/"auto"</code> was used, since a smooth term can't be summarised by a single coefficient).
<code>submodel</code>	Which submodel to plot: "mu" (location, default) or "sigma" (scale).
<code>top_n</code>	Maximum number of variables shown in the coefficient path plot. Variables are chosen by selection frequency.
<code>scale</code>	Coefficient scale for <code>type = "path"</code> . "original" (default) shows coefficients on the original predictor scale for both submodels. For the scale submodel "original" means the additive effect on $\log \sigma$ per unit change in the original z_j . "standardized" shows coefficients on the standardised predictor scale.
<code>variable</code>	Required when <code>type = "partial"</code> : the name of the predictor (in submodel) whose fitted partial effect to plot.
<code>...</code>	Additional graphical arguments passed to base plotting functions.

Details

The path plot shows cumulative coefficient updates over boosting iterations. The frequency plot shows how often each predictor was selected. The partial-effect plot shows a single variable's fitted contribution to the (standardised) linear predictor across its observed range – a straight line for a variable fit entirely by the linear base learner, or a curve for one that received spline steps.

Value

The input object `x`, invisibly.

Examples

```

set.seed(123)
X <- data.frame(x1 = rnorm(80), x2 = rnorm(80), x3 = rnorm(80))
Z <- data.frame(z1 = rnorm(80), z2 = rnorm(80))
y <- 1 + 2 * X$x1 - X$x3 + exp(0.4 * Z$z1) * rnorm(80)

fit <- boost_gaussian(X, Z, y, mstop = 80)

plot(fit, type = "path",      submodel = "mu")
plot(fit, type = "frequency", submodel = "mu")
plot(fit, type = "path",      submodel = "sigma")
plot(fit, type = "frequency", submodel = "sigma")

```

plot.cv_boost_gaussian

Plot-method for CV Function "cv_boost_gaussian"

Description

Plot-method for CV Function "cv_boost_gaussian"

Usage

```
## S3 method for class 'cv_boost_gaussian'
plot(x, ...)
```

Arguments

x	object of class "cv_boost_gaussian"
...	other parameters for the plot() function

predict.boost_dist

Predict from a Distributional Boosting Model

Description

Computes fitted or predicted values from a boost_gamma, boost_poisson, or boost_binomial object (all of class "boost_dist"). Generalizes [predict.boost_gaussian](#)'s reconstruction logic to any number of distribution parameters.

Usage

```
## S3 method for class 'boost_dist'
predict(
  object,
  newdata_X = NULL,
  newdata_Z = NULL,
  newdata = NULL,
  mstop = NULL,
  what = NULL,
  ...
)
```

Arguments

object	A fitted object inheriting from class "boost_dist".
newdata_X	Optional data frame or matrix of predictors for the family's first parameter (e.g. the mean). If omitted (together with newdata_Z and newdata), in-sample fitted values are returned.
newdata_Z	Optional data frame or matrix of predictors for the family's second parameter, if it has one (e.g. Gamma's shape). Must be NULL for single-parameter families (Poisson, Binomial).
newdata	Optional single data frame with the original variable names, used when object was fit via the formula interface.
mstop	Optional non-negative integer number of boosting rounds to use; defaults to the full fitted model.
what	Which parameter to return predictions for (one of object\$parameters), or "all" for a data frame with one column per parameter. Defaults to the family's only parameter for single-parameter families, or "all" otherwise.
...	Further arguments, currently ignored.

Value

A numeric vector (single parameter) or a data frame with one column per parameter (what = "all").

Examples

```
set.seed(123)
X <- data.frame(x1 = rnorm(80))
y <- rpois(80, lambda = exp(0.3 + 0.5 * X$x1))
fit <- boost_poisson(X, y, mstop = 60)
predict(fit)
predict(fit, mstop = 20)
```

predict.boost_gaussian

Predict from a Distributional Gaussian Boosting Model

Description

Computes fitted or predicted values from a boost_gaussian object. Both the location ($\hat{\mu}$) and the scale ($\hat{\sigma}$) can be returned.

Usage

```
## S3 method for class 'boost_gaussian'
predict(
  object,
  newdata_X = NULL,
```

```

newdata_Z = NULL,
newdata = NULL,
mstop = NULL,
what = c("mu", "sigma", "both"),
...
)

```

Arguments

<code>object</code>	A fitted object of class "boost_gaussian".
<code>newdata_X</code>	Optional data frame or matrix of location predictors. Must contain all columns used in X at fit time, matched by name. If omitted (together with <code>newdata_Z</code> and <code>newdata</code>), in-sample fitted values are returned.
<code>newdata_Z</code>	Optional data frame or matrix of scale predictors. Must contain all columns used in Z at fit time, matched by name. If omitted (together with <code>newdata_X</code> and <code>newdata</code>), in-sample fitted values are returned.
<code>newdata</code>	Optional single data frame with the original variable names, used when <code>object</code> was fit via the formula interface of <code>boost_gaussian</code> . An alternative to supplying <code>newdata_X/newdata_Z</code> directly.
<code>mstop</code>	Optional non-negative integer: number of boosting rounds to use. Defaults to the full fitted model. <code>mstop = 0</code> returns the intercept-only predictions. Any value in <code>0:object\$mstop</code> is allowed, so predictions can be obtained at an earlier stopping round both in-sample and out-of-sample. A "round" counts the same way it did during fitting: for <code>method = "cyclic"</code> each round may update both sub-models (or only one, once the other's separate budget is exhausted); for <code>method = "noncyclic"</code> each round updates exactly one submodel.
<code>what</code>	Character string specifying what to return: "mu" (default) for location predictions, "sigma" for scale predictions (on the original sigma scale), or "both" for a data frame with columns <code>mu</code> and <code>sigma</code> .
<code>...</code>	Further arguments, currently ignored.

Details

Predictions are reconstructed by accumulating the stored per-iteration updates (intercept plus slope) on the standardised predictor scale. For out-of-sample data the supplied `newdata_X` / `newdata_Z` are standardised with the training means and standard deviations; for in-sample prediction the stored standardised training design is used. Scale predictions are returned on the original σ scale (i.e. exp of the linear predictor on the log scale).

Value

A numeric vector (for `what = "mu"` or `"sigma"`) or a data frame with columns `mu` and `sigma` (for `what = "both"`).

Examples

```

set.seed(123)
X <- data.frame(x1 = rnorm(50), x2 = rnorm(50))

```

```

Z <- data.frame(z1 = rnorm(50))
y <- 1 + 2 * X$x1 + exp(0.3 * Z$z1) * rnorm(50)

fit <- boost_gaussian(X, Z, y, mstop = 50)

# In-sample predictions (full model)
predict(fit)
predict(fit, what = "sigma")
predict(fit, what = "both")

# Early stopping via mstop, in-sample and out-of-sample
predict(fit, mstop = 10)
predict(fit, newdata_X = X[1:3, ], newdata_Z = Z[1:3, ], drop = FALSE)
predict(fit, newdata_X = X[1:3, ], newdata_Z = Z[1:3, ], drop = FALSE,
        mstop = 10)
predict(fit, newdata_X = X[1:3, ], newdata_Z = Z[1:3, ], drop = FALSE,
        what = "both")

```

predict.cv_boost_gaussian

predict-method for CV Function "cv_boost_gaussian"

Description

predict-method for CV Function "cv_boost_gaussian"

Usage

```

## S3 method for class 'cv_boost_gaussian'
predict(object, newdata_X = NULL, newdata_Z = NULL, ...)

```

Arguments

object	object of class "cv_boost_gaussian"
newdata_X	new location predictor data
newdata_Z	new scale predictor data
...	further parameters for the prediction

Value

numeric prediction-vector

predict.cv_boost_grid *predict-method for CV Function "cv_boost_grid"*

Description

predict-method for CV Function "cv_boost_grid"

Usage

```
## S3 method for class 'cv_boost_grid'
predict(object, newdata_X = NULL, newdata_Z = NULL, ...)
```

Arguments

object	object of class "cv_boost_grid"
newdata_X	new location predictor data
newdata_Z	new scale predictor data
...	further parameters for the prediction

Value

numeric prediction-vector

print.boost_dist *Print a Distributional Boosting Model*

Description

Prints a compact overview of a fitted boost_gamma, boost_poisson, or boost_binomial object (all of class "boost_dist"), generalizing [print.boost_gaussian](#) to any number of distribution parameters.

Usage

```
## S3 method for class 'boost_dist'
print(x, digits = 4, ...)
```

Arguments

x	A fitted object inheriting from class "boost_dist".
digits	Number of digits used when printing numeric values.
...	Further arguments, currently ignored.

Value

The input object x, invisibly.

```
print.boost_gaussian
```

Print a Distributional Gaussian Boosting Model

Description

Prints a compact overview of a fitted `boost_gaussian` object, including tuning parameters, predictor selection counts for both the location and scale submodels, and basic training-fit statistics.

Usage

```
## S3 method for class 'boost_gaussian'
print(x, digits = 4, ...)
```

Arguments

<code>x</code>	A fitted object of class "boost_gaussian".
<code>digits</code>	Number of digits used when printing numeric values.
<code>...</code>	Further arguments, currently ignored.

Value

The input object `x`, invisibly.

Examples

```
set.seed(123)
X <- data.frame(x1 = rnorm(50), x2 = rnorm(50))
Z <- data.frame(z1 = rnorm(50))
y <- 1 + 2 * X$x1 + exp(0.3 * Z$z1) * rnorm(50)

fit <- boost_gaussian(X, Z, y, mstop = 30)
print(fit)
```

```
print.cv_boost_gaussian
```

print-method for CV Function "cv_boost_gaussian"

Description

print-method for CV Function "cv_boost_gaussian"

Usage

```
## S3 method for class 'cv_boost_gaussian'
print(x, ...)
```

Arguments

x	object of class "cv_boost_gaussian"
...	other parameters for the print() function

```
print.cv_boost_grid    print-method for CV Function "cv_boost_grid"
```

Description

print-method for CV Function "cv_boost_grid"

Usage

```
## S3 method for class 'cv_boost_grid'
print(x, ...)
```

Arguments

x	object of class "cv_boost_grid"
...	other parameters for the print() function

```
print.summary.boost_dist
Print a Distributional Boosting Model Summary
```

Description

Print a Distributional Boosting Model Summary

Usage

```
## S3 method for class 'summary.boost_dist'
print(x, digits = 4, ...)
```

Arguments

x	An object of class "summary.boost_dist".
digits	Number of digits used when printing numeric values.
...	Further arguments, currently ignored.

Value

The input object x, invisibly.

```
print.summary.boost_gaussian
```

Print a Distributional Gaussian Boosting Model Summary

Description

Prints a formatted summary produced by `summary.boost_gaussian`.

Usage

```
## S3 method for class 'summary.boost_gaussian'
print(x, digits = 4, ...)
```

Arguments

<code>x</code>	An object of class "summary.boost_gaussian".
<code>digits</code>	Number of digits used when printing numeric values.
<code>...</code>	Further arguments, currently ignored.

Value

The input object `x`, invisibly.

Examples

```
set.seed(123)
X <- data.frame(x1 = rnorm(60), x2 = rnorm(60), x3 = rnorm(60))
Z <- data.frame(z1 = rnorm(60), z2 = rnorm(60))
y <- 1 + 2 * X$x1 - X$x3 + exp(0.4 * Z$z1) * rnorm(60)

fit <- boost_gaussian(X, Z, y, mstop = 50)
print(summary(fit))
```

```
print.summary.cv_boost_gaussian
```

extra print-method for summary-method of "cv_boost_gaussian"

Description

extra print-method for summary-method of "cv_boost_gaussian"

Usage

```
## S3 method for class 'summary.cv_boost_gaussian'
print(x, ...)
```

Arguments

x	object of class "summary.cv_boost_gaussian"
...	other parameters passed

```
print.summary.cv_boost_grid
```

extra print-method of summary-method of "cv_boost_grid"

Description

extra print-method of summary-method of "cv_boost_grid"

Usage

```
## S3 method for class 'summary.cv_boost_grid'
print(x, ...)
```

Arguments

x	object of class "summary.cv_boost_grid"
...	other parameters passed

```
summary.boost_dist
```

Summarize a Distributional Boosting Model

Description

Summarize a Distributional Boosting Model

Usage

```
## S3 method for class 'boost_dist'
summary(object, ...)
```

Arguments

object	A fitted object inheriting from class "boost_dist".
...	Further arguments, currently ignored.

Value

An object of class "summary.boost_dist".

summary.boost_gaussian

Summarize a Distributional Gaussian Boosting Model

Description

Computes a structured summary of a fitted boost_gaussian object, covering both the location (mean) and scale (standard deviation) submodels.

Usage

```
## S3 method for class 'boost_gaussian'
summary(object, ...)
```

Arguments

object	A fitted object of class "boost_gaussian".
...	Further arguments, currently ignored.

Details

Both location and scale coefficients are reported on the *original predictor scale*:

- **Location:** a coefficient β_j means one unit increase in the original x_j increases $\hat{\mu}$ by β_j .
- **Scale:** a coefficient γ_j means one unit increase in the original z_j changes $\log \hat{\sigma}$ by γ_j , i.e. multiplies $\hat{\sigma}$ by $\exp(\gamma_j)$. The linear predictor is always on the log σ scale to ensure $\hat{\sigma} > 0$.

Residual diagnostics are based on the raw residuals $y - \hat{\mu}$.

Value

An object of class "summary.boost_gaussian".

Examples

```
set.seed(123)
X <- data.frame(x1 = rnorm(60), x2 = rnorm(60), x3 = rnorm(60))
Z <- data.frame(z1 = rnorm(60), z2 = rnorm(60))
y <- 1 + 2 * X$x1 - X$x3 + exp(0.4 * Z$z1) * rnorm(60)

fit <- boost_gaussian(X, Z, y, mstop = 50)
summary(fit)
```

```
summary.cv_boost_gaussian
```

summary-method for CV Function "cv_boost_gaussian"

Description

summary-method for CV Function "cv_boost_gaussian"

Usage

```
## S3 method for class 'cv_boost_gaussian'
summary(object, ...)
```

Arguments

object	An object of class "cv_boost_gaussian"
...	other parameters for the summary() function

Value

list of values for model summary

```
summary.cv_boost_grid
```

summary-method for CV Function "cv_boost_grid"

Description

summary-method for CV Function "cv_boost_grid"

Usage

```
## S3 method for class 'cv_boost_grid'
summary(object, ...)
```

Arguments

object	object of class "cv_boost_grid"
...	other parameters for the summary() function

Value

list with sorted grid results and best combination

Index

`boost_binomial`, [2](#)
`boost_gamma`, [3](#)
`boost_gaussian`, [2–4](#), [5](#), [8](#), [16](#)
`boost_poisson`, [7](#)

`cv_boost_gaussian`, [9](#)
`cv_boost_grid`, [10](#)

`generate_folds`, [11](#)

`plot.boost_dist`, [11](#)
`plot.boost_gaussian`, [11](#), [12](#), [12](#)
`plot.cv_boost_gaussian`, [14](#)
`predict.boost_dist`, [14](#)
`predict.boost_gaussian`, [14](#), [15](#)
`predict.cv_boost_gaussian`, [17](#)
`predict.cv_boost_grid`, [18](#)
`print.boost_dist`, [18](#)
`print.boost_gaussian`, [18](#), [19](#)
`print.cv_boost_gaussian`, [19](#)
`print.cv_boost_grid`, [20](#)
`print.summary.boost_dist`, [20](#)
`print.summary.boost_gaussian`, [21](#)
`print.summary.cv_boost_gaussian`, [21](#)
`print.summary.cv_boost_grid`, [22](#)

`summary.boost_dist`, [22](#)
`summary.boost_gaussian`, [21](#), [23](#)
`summary.cv_boost_gaussian`, [24](#)
`summary.cv_boost_grid`, [24](#)